C Programming - Assignment 1 (CO1)

Task 9: Implement and Test the Billing Program

Grama Jyothi Rural Electricity Co-operative

Bloom's Level: K3 - Apply | Marks: 4

Student name	Sk.Thohith
Register number	1912260555
Repository	CSA02-<RegisterNumber>-Assignment1 (folder: Task09)
GitHub link	_____
Compiler	GCC 13.3.0 (Ubuntu 24.04), also run on an online C compiler (see Section 5)
Compile command	gcc -Wall -Wextra -std=c11 -o billing main.c

Contents

- 1. Aim and scope
- 2. Design: data, constants and rules
- 3. Pseudocode and UML activity diagram
- 4. Source code (main.c)
- 5. Execution evidence (screenshot)
- 6. Test plan and results (17 cases)
- 7. Hand-checked set of five bills
- 8. Findings: one design limit found by testing, and its fix
- 9. Alternative concept: slabs in arrays with a loop (time and space analysis)
- 10. Conclusion

1. Aim and scope

To implement the complete billing program for the co-operative and to test it so that every bill is correct to the paisa. The program reads the previous and current meter readings, finds the units consumed (including a meter that has rolled over from 65535 to 0), applies the three-slab tariff plus the fixed charge, and prints the bill in rupees and paise. Invalid input is detected and rejected. The testing covers normal bills in each slab, boundary cases (0, 100 and 200 units), roll-over, invalid input, and a set of five bills whose total was calculated by hand.

2. Design: data, constants and rules

Item	C type	Reason
Meter reading (register)	typedef unsigned short meter_t	The register is 16-bit and never negative (0 to 65535). Arithmetic on this type wraps modulo 65536, which is exactly the roll-over behaviour.
Readings typed by the user	unsigned int	Read with %u so the full range can be checked against METER_MAX before narrowing to meter_t.
Units consumed	unsigned int	At most 65535, never negative.
Money (bill, charges)	long long, in paise	Integer paise removes every rounding error. long long also keeps district-scale totals safe.
Tariff values	#define constants	SLAB1_UNITS 100, SLAB2_UNITS 200, RATE1_PAISE 0, RATE2_PAISE 225, RATE3_PAISE 450, FIXED_CHARGE_PAISE 5000LL, METER_MAX 65535u. A tariff revision edits these lines only.

Rules every correct bill must satisfy (used to design the tests):

- The bill is never negative, and is at least the fixed charge of Rs 50.00.
- Total = fixed charge + slab-1 charge + slab-2 charge + slab-3 charge.
- The bill never decreases when units increase (monotonic).
- Roll-over: units = (current - previous) modulo 65536.
- Rupees and paise are printed from one integer: bill / 100 and bill % 100, with the paise printed in two digits (%02lld).

Why units are 256 after roll-over. For 65400 to 120, current - previous = 120 - 65400 = -65280. Cast back to the 16-bit unsigned type this becomes -65280 + 65536 = 256, which is the true distance the meter travelled (65400 to 65535 is 135 units, one more unit to reach 0, then 120 more: 135 + 1 + 120 = 256).

3. Pseudocode and UML activity diagram

```
START
  READ previous, current
  IF reading two numbers failed THEN print "Invalid input."  STOP(1)
  IF previous > 65535 OR current > 65535 THEN
       print "Readings must be between 0 and 65535."  STOP(1)
  units <- (unsigned 16-bit) (current - previous)
  bill  <- compute_bill(units)
  PRINT units, bill / 100, bill MOD 100 (two digits)
STOP(0)

FUNCTION compute_bill(units)          // result in paise
  energy <- 0
  IF units > 200 THEN energy <- energy + (units - 200) * 450 ; units <- 200
  IF units > 100 THEN energy <- energy + (units - 100) * 225 ; units <- 100
  energy <- energy + MIN(units, 100) * 0
  RETURN 5000 + energy
```

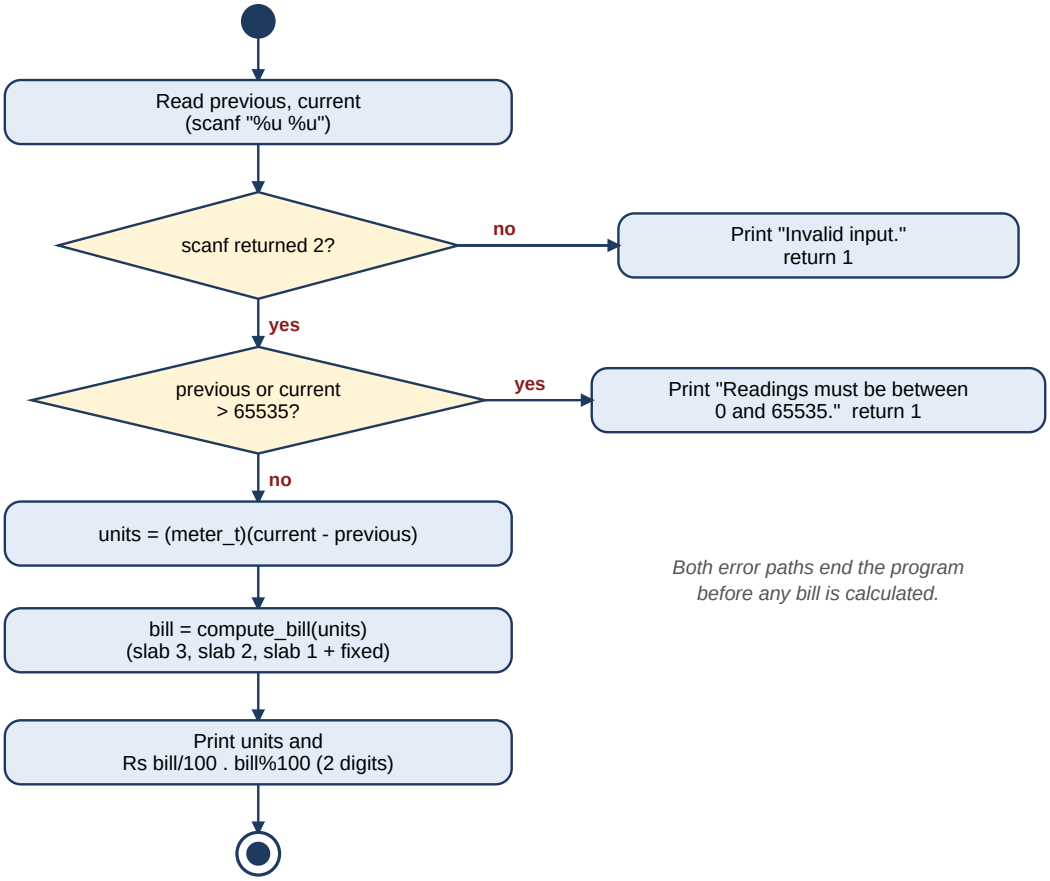


Figure 1: UML activity diagram of main(). The error exits happen before any calculation.

4. Source code (main.c)

The code below is the program run in the screenshot of Section 5. It is saved in the repository as Task09/main.c.

```
/*
 * Task 9 - Implement and Test the Billing Program
 * Grama Jyothi Rural Electricity Co-operative
 * All money is held in integer paise (long long), so no rounding error.
 */
#include <stdio.h>
#include <limits.h>

/* ---- Tariff constants: a revision is made here, in one place only ---- */
#define SLAB1_UNITS         100      /* first 100 units            */
#define SLAB2_UNITS         200      /* units 101 to 200           */
#define RATE1_PAISE         0        /* first slab is free         */
#define RATE2_PAISE         225      /* Rs 2.25 per unit           */
#define RATE3_PAISE         450      /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE  5000LL   /* Rs 50 fixed charge         */
#define METER_MAX           65535u   /* 16-bit register maximum    */

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

/* Units consumed; unsigned 16-bit arithmetic absorbs the roll-over. */
unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}

/* Returns the bill in paise (fixed charge + energy charge). */
long long compute_bill(unsigned int units)
{
    long long energy = 0;

    if (units > SLAB2_UNITS)                    /* slab 3: above 200 */
    {
        energy += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS)                    /* slab 2: 101..200  */
    {
        energy += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }
    energy += (long long)(units < SLAB1_UNITS
                        ? units : SLAB1_UNITS)   /* slab 1: 0..100 */
              * RATE1_PAISE;

    return FIXED_CHARGE_PAISE + energy;
}

int main(void)
{
    unsigned int previous, current;

    printf("Enter previous and current readings: ");

    if (scanf("%u %u", &previous, &current) != 2)
    {
        printf("Invalid input.\n");
        return 1;
    }

    if (previous > METER_MAX || current > METER_MAX)
    {
        printf("Readings must be between 0 and 65535.\n");
        return 1;
    }

    unsigned int units = consumption(previous, current);
    long long bill = compute_bill(units);

    printf("Units consumed: %u\n", units);
    printf("Energy and fixed charges included.\n");
    printf("Total bill: Rs %lld.%02lld\n",
           bill / 100, bill % 100);

    return 0;
}
```

5. Execution evidence

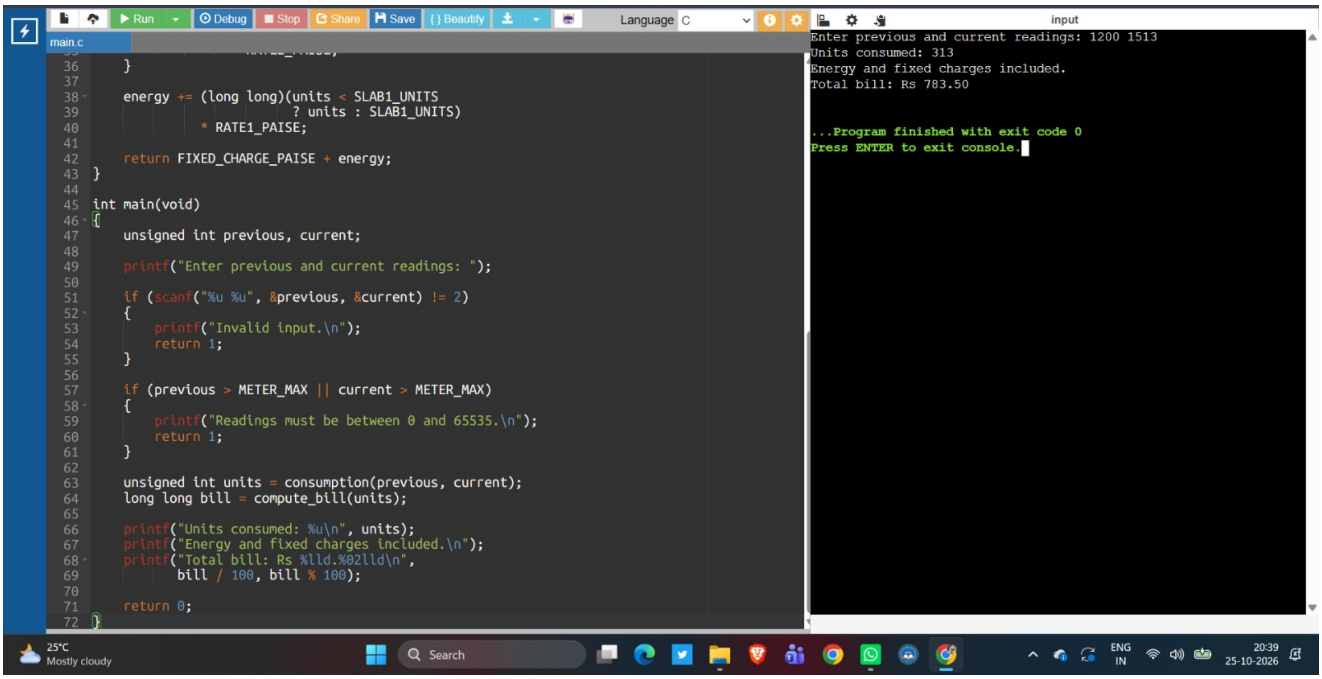


Figure 2: Program run in the online compiler (main.c on the left, console on the right). Input 1200 1513 gives 313 units and Rs 783.50.

Check by hand. 313 units: first 100 units free = Rs 0; next 100 units x Rs 2.25 = Rs 225.00; remaining 113 units x Rs 4.50 = Rs 508.50. Energy charge = Rs 733.50; adding the fixed charge of Rs 50.00 gives **Rs 783.50**, which matches the program output.

Compile command and warnings. `gcc -Wall -Wextra -std=c11 -o billing main.c` compiles with no warnings and no errors.

6. Test plan and results

Each test feeds one line of input to the program. The expected value comes from the hand calculation of the tariff (not from the program). Results were produced by running the compiled program with the input redirected from the test list, so every row is a real execution.

ID	Case	Input	Expected	Actual	Result
T1	Normal - slab 3	1200 1513	Units consumed: 313 | Rs 783.50	Units consumed: 313 | Rs 783.50	PASS
T2	Normal - slab 1	1000 1050	Units consumed: 50 | Rs 50.00	Units consumed: 50 | Rs 50.00	PASS
T3	Normal - slab 2	1000 1150	Units consumed: 150 | Rs 162.50	Units consumed: 150 | Rs 162.50	PASS
T4	Normal - slab 3	1000 1350	Units consumed: 350 | Rs 950.00	Units consumed: 350 | Rs 950.00	PASS
T5	Boundary - exactly 100	500 600	Units consumed: 100 | Rs 50.00	Units consumed: 100 | Rs 50.00	PASS
T6	Boundary - exactly 200	500 700	Units consumed: 200 | Rs 275.00	Units consumed: 200 | Rs 275.00	PASS
T7	Boundary - zero units	800 800	Units consumed: 0 | Rs 50.00	Units consumed: 0 | Rs 50.00	PASS
T8	Boundary - 201 units	500 701	Units consumed: 201 | Rs 279.50	Units consumed: 201 | Rs 279.50	PASS
T9	Roll-over 65400 -> 120	65400 120	Units consumed: 256 | Rs 527.00	Units consumed: 256 | Rs 527.00	PASS
T10	Roll-over 65535 -> 0	65535 0	Units consumed: 1 | Rs 50.00	Units consumed: 1 | Rs 50.00	PASS

ID	Case	Input	Expected	Actual	Result
T11	Invalid - letters (1st)	abc 100	Invalid input.	Invalid input.	PASS
T12	Invalid - letters (2nd)	100 xyz	Invalid input.	Invalid input.	PASS
T13	Invalid - negative previous	-5 100	Readings must be between 0 and 65535.	Readings must be between 0 and 65535.	PASS
T14	Invalid - negative current	100 -5	Readings must be between 0 and 65535.	Readings must be between 0 and 65535.	PASS
T15	Invalid - above 65535	70000 100	Readings must be between 0 and 65535.	Readings must be between 0 and 65535.	PASS
T16	Invalid - empty input	(empty line)	Invalid input.	Invalid input.	PASS
T17	Current < previous, no roll-over	5000 4000	Rejected (not a valid monthly reading)	Units consumed: 64536 | Rs 289787.00	FAIL

Summary: 16 of 17 cases pass. Test T17 exposes a design limit, discussed in Section 8. Note on T13 and T14: scanf with %u accepts a minus sign and wraps -5 to 4294967291, which the range check then rejects, so negative readings are caught by the same test as readings above 65535.

7. Hand-checked set of five bills

Five households were chosen to cover every slab and a roll-over. The expected column is calculated by hand; the actual column is the program output for the same readings.

House	Previous	Current	Units	Working (paise)	Expected	Program
H1	1000	1000	0	5000	Rs 50.00	Rs 50.00
H2	2000	2100	100	5000 + 100x0	Rs 50.00	Rs 50.00
H3	3000	3150	150	5000 + 50x225 = 16250	Rs 162.50	Rs 162.50
H4	65400	120	256	5000 + 100x225 + 56x450 = 52700	Rs 527.00	Rs 527.00
H5	1200	1513	313	5000 + 100x225 + 113x450 = 78350	Rs 783.50	Rs 783.50
Total			**819**	5000+5000+16250+52700+78350 = 157300	**Rs 1573.00**	**Rs 1573.00**

The total of the five program outputs is 50.00 + 50.00 + 162.50 + 527.00 + 783.50 = Rs 1573.00, equal to the hand total. Because every bill is an integer number of paise, the sum is exact, with no drift.

8. Findings: one design limit found by testing, and its fix

Observation (T17). For previous = 5000 and current = 4000 the program prints 64536 units and Rs 289787.00. A 16-bit register cannot tell a genuine roll-over from a wrongly typed or mis-read value, because (current - previous) modulo 65536 is always defined. The assignment lists this input as invalid, so the base program does not meet that requirement by itself.

Fix. Add one named constant for the largest plausible monthly consumption and reject units above it (this is an assumption the co-operative must set; 5000 units is used here). Roll-over cases stay valid because their true distance is small (256 for 65400 to 120).

```
#define MAX_MONTHLY_UNITS   5000u   /* assumed upper limit for one household-month */
...
    unsigned int units = consumption(previous, current);

    if (units > MAX_MONTHLY_UNITS)
    {
        printf("Implausible consumption (%u units): check the readings.\n", units);
        return 1;
    }
```

With this check (file main_v2.c), T17 prints: `Implausible consumption (64536 units): check the readings.` and exits with code 1. The other 16 tests give identical results in main_v2.c, so the change does not disturb any valid bill.

Test	Input	main.c (as submitted)	main_v2.c (with plausibility check)
T17	5000 4000	Units 64536, Rs 289787.00 (accepted)	Rejected, exit code 1
T9	65400 120	256 units, Rs 527.00	256 units, Rs 527.00 (unchanged)

9. Alternative concept and complexity analysis

Question. If the slab limits and rates were stored in two arrays and the bill computed by a loop over the slabs, instead of fixed if statements, what would the solution be?

Solution. The tariff becomes data: `limit[] = {100, 200, max}` and `rate[] = {0, 225, 450}`. One loop charges each slab with the units that fall inside it:

```
bill = FIXED
lo = 0
FOR i = 0 TO s-1 WHILE units > lo
    hi   = MIN(limit[i], units)
    bill = bill + (hi - lo) * rate[i]
    lo   = limit[i]
RETURN bill
```

The loop version (alt_loop.c) was tested against the if-version for every possible consumption from 0 to 65535 units: **0 mismatches**.

	Fixed if statements (main.c)	Arrays + loop
Time per bill	O(1): 2 tests and at most 3 multiplications, independent of the number of slabs	O(s) for s slabs (here s = 3, so about the same order, but it grows with s)
Extra space	O(1): no tables, the constants are compiled in	O(s): two arrays of s entries (36 bytes here: 3 x 4 for limits + 3 x 8 for rates)
Measured, 800,000 bills	about 3.0 to 3.2 ms per run (50 runs averaged)	about 6.3 to 7.5 ms per run (50 runs averaged)
Ease of change	Adding a slab means editing code and recompiling	Adding a slab means adding one entry to each array; the loop is unchanged, and the arrays can later be filled from a file

The timings were taken with gcc -O0 on the machine used to prepare this report (benchmark in alt_loop.c). Absolute values depend on the computer, so re-run alt_loop.c on your own system for the figures you quote; the ratio (loop roughly twice as slow here) is the expected pattern for a loop with a test and array reads per iteration. **Which design is easier to change:** the array design. **Which is faster for this tariff:** the if design. Both are far below any practical limit: 800,000 bills take only a few milliseconds, so for this co-operative the loop's flexibility costs nothing noticeable, and it also leads naturally to the run-time tariff file needed in Task 12.

10. Conclusion

- The program meets the expected output of the screenshot case: 1200 and 1513 give 313 units and Rs 783.50.
- Every slab, the boundaries at 100 and 200 units, zero units and both roll-over cases give the hand-calculated paise value. The five-bill total is exactly Rs 1573.00.
- Invalid input (letters, negative numbers, values above 65535, empty input) is rejected with a message and exit code 1 before any bill is computed.
- Testing found one real weakness: a current reading lower than the previous one cannot be separated from a roll-over by the register alone. A named constant MAX_MONTHLY_UNITS fixes this with a three-line change.
- Storing money as integer paise in long long made every result exact, and keeping all tariff values as macros means a revision is a one-line edit.

Files for the repository (folder Task09): main.c, main_v2.c, alt_loop.c, tests.py (test driver), results screenshot (21059.jpg), README.md entry with the compile command above.